

MODULE 7 · LEVEL 300

Maintenance & Support: Operating with AI



Where systems spend most of their life - incidents, logs, monitoring, security and tech-debt, with GitHub Copilot

Day 3 · 90 minutes · The final module · Novnex



Where we are, and today's shape

⌚ approx 2 min

We've shipped. Now we operate and sustain - and this is the final module, so we'll also tie the whole lifecycle together.



Framing

Why maintenance is where AI quietly wins, and the map.

~18 min



Maintenance with Copilot

Incidents, logs, monitoring, security, legacy, debt, docs.

~49 min



Apply & close

By track, the full SDLC, what to do next, and a wrap.

~23 min

Most of a system's life is spent here - so this is where small, steady AI help adds up to the most.



Learning objectives

 approx 2 min

By the end of this module you will be able to:



Resolve faster

Use AI to triage incidents and analyse logs to find root cause quickly.



Operate proactively

Cut alert noise and anticipate failures before users feel them.



Sustain quality

Keep security, dependencies and docs current as the system evolves.



Operate safely

Keep humans approving production actions and PII out of prompts.



Maintenance: where AI quietly wins

⌚ approx 4 min



The unglamorous majority

- **Most of the life:**
 - a system spends far longer being run than being built.
- **Where toil lives:**
 - on-call, log-diving, ticket triage, dependency bumps.
- **Small help, huge sum:**
 - minutes saved per incident, thousands of times.
- **Under-tooled:**
 - exactly the work that never gets the good tools.



What good looks like

- Mean-time-to-resolve drops - not just mean-time-to-code
- Noise falls; signal rises; on-call is humane
- Security and docs stay current instead of rotting
- People spend less time firefighting, more improving



Where AI helps across maintenance

 approx 4 min



Incidents

Triage, chatbots, guided resolution.



Log analysis

Find the needle; spot anomalies.



Monitoring

Smarter alerts, less noise.



Security upkeep

Scanning, Autofix, dependencies.



Legacy & debt

Understand and modernise old code.



Documentation

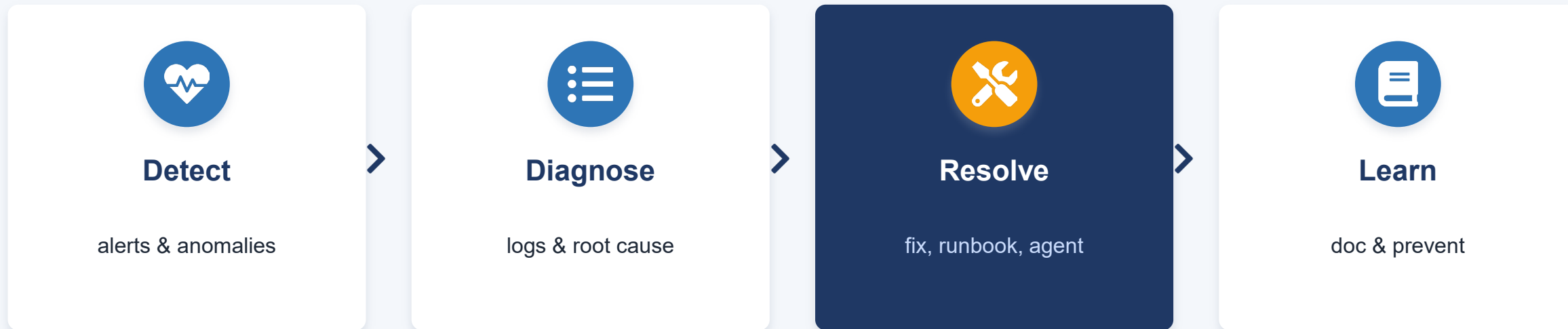
Keep it current, automatically.

We'll take these in order. The rule holds to the end: AI assists the operator; a human still owns production.



The operate loop - where Copilot plugs in

⌚ approx 4 min



Detect, diagnose, resolve, learn - AI accelerates each turn of the loop, and the loop keeps getting shorter.

PART 2

Maintenance with Copilot

Incidents, logs, monitoring, prediction, security, legacy, debt and docs - with real examples.

approx 49 min





Incident management

⌚ approx 5 min

Triage in seconds

```
ALERT: 5xx spike on /api/login (P1)
> summarise: what changed, blast radius?
Copilot: deploy #2.4 30m ago; reset flow;
        errors from the rate-limiter config.
> suggest next steps from the runbook
-> roll back #2.4; verify; open incident
```

Faster to resolution

- **Summarise fast:**
 - what changed, what's affected, likely cause.
- **Guided resolution:**
 - pull the right runbook and next steps.
- **Auto-resolve common ones:**
 - chatbots handle known issues.
- **Human owns the call:**
 - you approve the rollback or fix.



Log analysis with AI

🕒 approx 5 min

Find the needle

```
120,412 lines in the last hour...  
> cluster the errors and rank by impact  
1) NullRef in ResetService (812x) new!  
2) slow query > 2s on tickets (233x)  
3) timeout to KYC (41x)  
-> #1 correlates with deploy #2.4
```

Signal from noise

- **Cluster & rank:**
 - turn a million lines into a short list.
- **Spot the 'new':**
 - what started, and when.
- **Correlate:**
 - link errors to a deploy or a config change.
- **Plain-language:**
 - ask questions instead of writing regex.



Monitoring & alerting

 approx 4 min



Smarter signals

- **Anomaly detection:**
 - flag the abnormal, not just fixed thresholds.
- **Alert triage:**
 - group and summarise a storm into one story.
- **Draft the monitors:**
 - 'alert when p95 latency doubles for 5 min'.
- **Explain the alert:**
 - what it means and where to look.



Fight alert fatigue

- Too many alerts is the same as none
- AI tuning helps - but you decide what pages a human
- Alert on user pain, not on every wiggle
- Quieter, sharper alerting keeps on-call humane



Predictive maintenance

⌚ approx 4 min

See it before it breaks

```
trend: disk on db-2 -> full in ~6 days  
trend: error rate creeping since Tue  
cert: portal TLS expires in 9 days  
-> open proactive tickets + owners
```

From reactive to ahead

- **Trend spotting:**
 - catch slow creeps humans miss day to day.
- **Capacity & expiry:**
 - disks, certs, quotas - before they bite.
- **Proactive tickets:**
 - fix on a Tuesday, not at 3am.
- **Fewer 3am pages:**
 - the best incident is the one avoided.



Ongoing security upkeep

⌚ approx 4 min

Security never sleeps

```
Dependabot: 3 vulnerable deps  
-> agent opens PRs to bump + test  
Code scanning: XSS in search (Autofix)  
Secret scanning: token in old commit
```

Keep it hardened

- **Dependencies:**
 - Dependabot flags; agents draft the bump PRs.
- **Autofix:**
 - scanning alerts become suggested fixes.
- **Secrets:**
 - AI catches leaked credentials to rotate.
- **You review:**
 - security fixes still get a human check.



Understand & modernise legacy code

⌚ approx 4 min



Tame the old code

- **Explain it:**
 - turn a mystery module into something readable.
- **Document it:**
 - generate the docs the original author never wrote.
- **Add tests first:**
 - characterisation tests before you change anything.
- **Modernise safely:**
 - refactor in small steps behind those tests.



Respect the risk

- Legacy code often encodes hard-won business rules
- Verify AI's explanation before you trust it
- Tests are your safety net for any change
- [Small steps beat a big rewrite you can't review](#)



Tech-debt & dependency upkeep

 approx 4 min



Chip at the debt

Delegate well-scoped clean-ups to the cloud agent.



Stay patched

Dependabot plus agent PRs keep versions current.



Tested by default

Upkeep changes ride the suite - green or it doesn't merge.



Prioritise

Fix the debt that actually slows delivery or raises risk.



Documentation that stays current

⌚ approx 3 min



READMEs & guides

Generate and refresh docs as the code changes.



Runbooks

Turn a resolved incident into a reusable runbook.



ADRs & decisions

Keep the 'why' current as the design evolves.



Onboarding

Explainers that get new joiners productive faster.

Docs rot because updating them is a chore - AI removes the chore, so the docs stay worth reading.



Custom agents for operations

⌚ approx 4 min

Extend Copilot to your stack

```
agents (Markdown-defined):  
  - PagerDuty incident responder  
  - Observability expert (your tool)  
  - Runbook & escalation assistant  
+ partner integrations (e.g. Jira)
```

Ops is extensible

- **Domain agents:**
 - teach Copilot your ops tools and procedures.
- **Where it's headed:**
 - ops is extensible today, not turnkey.
- **Grounded:**
 - in your runbooks, dashboards and history.
- **Still supervised:**
 - agents assist; humans hold prod.



Live demo: incident to fix

 approx 6 min

Watch a loop close

- An alert fires; we ask Copilot to summarise
- It analyses the logs and finds the likely cause
- It correlates the error with a recent deploy
- We open an issue; the cloud agent drafts a fix
- We review, approve - and capture a runbook

Flow

- **1.**
 - Summarise the alert.
- **2.**
 - Analyse the logs.
- **3.**
 - Correlate to a deploy.
- **4.**
 - Agent drafts a fix.
- **5.**
 - Review; write a runbook.



Responsible operations with AI

⌚ approx 4 min



Auto-acting in prod

Letting an agent change production alone.

Guard: Humans approve prod actions; agents assist, never act unattended.



PII in prompts/logs

Pasting customer data while debugging.

Guard: Scrub and redact; content exclusion; never paste real PII.



Over-trusting analysis

Acting on an AI root-cause without checking.

Guard: Treat it as a hypothesis; confirm before a fix-forward.



No audit trail

Untracked AI-driven changes.

Guard: Log actions; keep audit trails; changes go through review.

PART 3

Apply & close

By track, the whole lifecycle tied together, what to do next - and a wrap.

approx 23 min





Maintenance with Copilot - by track

 approx 4 min



Web / Drupal

Triage portal errors, patch modules and deps, keep docs and runbooks current.



Data & Analytics

Monitor pipelines and freshness; diagnose failed jobs; data-quality alerts.



Power BI

Watch dataset refresh and performance; catch broken measures and reports early.



RPA

Handle bot exceptions and drift; agentic self-healing; keep process docs current.



Anti-patterns to avoid

 approx 3 min

Don't

- Act on a plausible AI diagnosis unchecked
- Let an agent remediate prod unattended
- Paste customer data into a chat to debug
- Chase every alert instead of the painful ones
- Let AI-driven changes skip the audit trail

Do

- Confirm the evidence before you fix forward
- Keep humans approving production actions
- Turn resolved incidents into runbooks
- Fix the debt that actually slows you down
- Redact PII before it reaches a prompt



One platform, the whole lifecycle

 approx 5 min

What we've done all week: the same agentic Copilot platform, applied end to end - each phase feeding the next.



Requirements

Notes to testable issues & specs. (M2)



Design

Architecture, diagrams, ADRs. (M3)



Development

Build to the design; delegate. (M4)



Testing

Trustworthy tests from criteria. (M5)



Deployment

Safe pipelines & rollouts. (M6)



Maintenance

Operate, triage, sustain. (M7)

The thread through all of it: AI proposes and accelerates; humans decide and stay accountable.



Where you go from here

approx 4 min



Start small

Pick one phase and one team; get a real win before scaling.



Measure your baseline

Cycle time, review time, defect escape - prove it with your data.



Set the guardrails

Policies, data boundaries, review habits - before you scale up.



Build shared context

Spaces and instructions so Copilot fits your org, not the internet.



Discussion & reflection

⌚ approx 3 min

Talk to the person next to you - three minutes:

- Across the whole week, which one change will you take back to your team first?
- What's the biggest thing that has to be true for AI to work well in your org?

Start small

Measure

Guardrails

Shared context

Culture

We'll take a few answers before we close.



That's a wrap - thank you

- You took one platform - GitHub Copilot - across the entire SDLC.
- From messy notes, to a design, to working, tested, shipped, and operated software.
- The tools will keep changing; the discipline won't - engineer the context, and review the output.
- AI proposes and accelerates. You decide, and you stay accountable.

On Monday

- Pick one workflow
- Ground it in a Space
- Measure the before/after
- Share what you learn

Go build - responsibly, and well.